Cuestionario de repaso — UML (defensa de tesis)

Guía de preguntas y respuestas basada en las decisiones reales tomadas durante el trabajo de nivelación UML de SYSCOMcli (Agendamiento, Consultorio, Ventas, Seguridad). No sustituye repasar los documentos `TALLER_V2_*.md` completos.

Índice de temas

1. UML — conceptos generales (9 preguntas)
2. Diagrama de casos de uso (9 preguntas)
3. Diagrama de clases (8 preguntas)
4. Diagrama de secuencia (8 preguntas)
5. Diagrama de actividades (8 preguntas)
6. StarUML y el archivo .uml (7 preguntas)
7. Diagrama de despliegue (4 preguntas)

---

1. UML — conceptos generales

1.1 ¿Qué es UML?

UML (Unified Modeling Language) es un lenguaje de modelado gráfico, estandarizado por la OMG, para especificar, construir y documentar la estructura y el comportamiento de un sistema de software. No es un método ni una notación de un solo diagrama: es un conjunto de diagramas (estructurales y de comportamiento) que representan distintas vistas del mismo sistema.

En la tesis se usó como herramienta de documentación y nivelación académica del sistema SYSCOMcli ya construido: no se diseñó el sistema a partir de los diagramas, sino que se modelaron los diagramas a partir del código y el flujo real, para el capítulo de análisis y diseño.

1.2 ¿Qué diferencia hay entre un diagrama estructural y uno de comportamiento?

Los estructurales (clases, componentes, despliegue) muestran la organización estática del sistema: qué elementos existen y cómo se relacionan. Los de comportamiento (casos de uso, secuencia, actividades, estados) muestran cómo interactúan esos elementos en el tiempo.

En este trabajo se usaron tres de comportamiento —Casos de Uso, Secuencia, Actividades— y uno estructural —Clases— por cada módulo (Agendamiento, Consultorio, Ventas, Seguridad).

1.3 Si el sistema ya estaba programado, ¿por qué hacer diagramas UML igual?

Porque el objetivo del capítulo de análisis y diseño no es generar código a partir del modelo (ingeniería directa), sino documentar formalmente el diseño que ya existe, con el rigor académico esperado en la cátedra — es un ejercicio de ingeniería inversa aplicado a fines de tesis.

Esto tuvo una consecuencia concreta: el profesor evaluador pidió que los diagramas de clase siguieran el formato académico legacy (capas boundary/control/entity genéricas) en vez del formato técnicamente preciso que reflejaba el código real 1:1 — porque lo que se evalúa es la convención UML enseñada en la cátedra, no la fidelidad byte a byte con el ORM.

1.4 ¿Qué es la arquitectura de tres capas boundary/control/entity y por qué aparece en los diagramas de clase?

Es un patrón clásico de análisis orientado a objetos (Jacobson): toda funcionalidad se modela con tres tipos de clase — boundary (interfaz con el usuario, ej. una pantalla), control (lógica/orquestación) y entity (datos persistentes). Es independiente de si el código real usa esa separación literal.

En SYSCOMcli, cada entidad (ej. Especialidad) tiene su `GUI_ESPECIALIDAD` (boundary), una clase `Conexion` compartida como control genérico, y `Especialidad` (entity) — siguiendo el formato legacy pedido, no el nombre real de las clases Python del proyecto.

1.5 ¿Qué es un actor en UML y cómo se decide quién es actor en este proyecto?

Un actor es cualquier entidad externa al sistema (persona, rol o sistema) que interactúa con él iniciando o participando de un caso de uso. No es necesariamente una persona física: puede ser un rol.

Regla aplicada en la tesis: el actor debe coincidir con el rol real de control de acceso en el código (el decorador `role_required(...)`), no con una intuición de quién "debería" hacerlo. Por eso, por ejemplo, nunca se usa Superadministrador como actor de los módulos de gestión clínica (Agendamiento/Consultorio/Ventas) — ese rol es de administración de la plataforma, no de la operación clínica del día a día.

1.6 ¿Qué es una piscina o carril (swimlane) y cuándo algo NO puede ser piscina?

Una piscina agrupa las acciones de un diagrama de actividades según quién las ejecuta. La regla usada en la tesis: una piscina representa un actor con agencia — alguien o algo capaz de ejecutar la acción — nunca un lugar físico ni un dato referencial pasivo.

Ejemplo aplicado: Consultorio no puede ser piscina porque es un dato que se asigna a un horario, no un actor. Sistema sí puede serlo cuando hay acciones automáticas reales en el código (jobs programados, validaciones automáticas).

1.7 ¿Composición y agregación son lo mismo?

No. Ambas son formas de asociación "todo–parte", pero con distinta dependencia de vida: en la agregación (rombo hueco) la parte puede existir independientemente del todo (es un catálogo que solo se consulta); en la composición (rombo relleno) la parte no tiene sentido ni puede existir sin el todo — muere en cascada con él.

Regla aplicada: cabecera→detalle (Orden→OrdenDetalle, Receta→RecetaDetalle, Remisión→RemisionDetalle) es composición. Detalle→catálogo (OrdenDetalle→TipoEstudio, RegistroDiagnostico→Diagnostico) es agregación.

1.8 Además de asociación, agregación y composición, ¿qué son dependencia y generalización?

La dependencia es la relación más débil: una clase usa a otra de forma puntual (ej. como parámetro o variable local de un método) sin mantener una referencia permanente — se dibuja con una flecha punteada. La generalización es la relación de herencia (clase padre/clase hija, "es un") — se dibuja con una flecha de línea llena y punta de triángulo hueco apuntando al padre.

En los diagramas de clase de esta tesis, la dependencia aparece entre las clases boundary/control y la clase `Conexion` (el boundary/control "depende de" o "usa" la conexión, sin ser dueño de ella) — no hay generalización, porque el formato legacy no modela jerarquías de herencia entre las clases GUI/Control/Entity.

1.9 ¿Qué es la transición en un diagrama de actividades?

Es la flecha que conecta dos nodos (estados, decisiones, sincronizaciones) e indica el paso de uno al siguiente cuando se completa la actividad de origen. En StarUML es el elemento `Transition` del toolbox — cada actividad, salvo el final, debe tener al menos una transición de salida, y cada una (salvo el inicio) al menos una de entrada; solo una `Decision` o una `Synchronization` pueden tener más de una transición de salida o de entrada al mismo tiempo.

---

2. Diagrama de casos de uso

2.1 ¿Qué es un caso de uso?

Es una unidad de funcionalidad que el sistema ofrece a un actor y que produce un resultado observable de valor para ese actor (ej. "Registrar Especialidad", "Registrar Consulta"). Se documenta con flujo básico, flujos alternativos, pre/postcondiciones y los archivos/tablas involucrados.

2.2 ¿Qué es una relación de inclusión (`<<include>>`) y dónde aparece en este proyecto?

Representa que un caso de uso base siempre incorpora, como parte obligatoria de su flujo, el comportamiento de otro caso de uso incluido — se factoriza comportamiento común (ej. validar sesión, registrar auditoría) sin duplicarlo en cada caso de uso.

En el archivo `.uml` legacy de la tesis existen casos de uso que están definidos en el modelo con su relación `<<include>>` ya armada pero sin ninguna vista dibujada en ningún diagrama — quedan "huérfanos" hasta que se les agrega una vista.

2.3 ¿Cómo está organizado el índice de casos de uso de la tesis (TALLER.MD)?

Por módulo: Agendamiento, Consultorio (clínico), Ventas y Seguridad, cada uno con sus referenciales (catálogos/mantenimientos simples) y sus movimientos (operaciones de negocio con flujo real, ej. Registrar Consulta, Registrar Factura). El índice final quedó en 52 casos de uso tras una corrección del profesor sobre el módulo de Seguridad (ver 2.5).

2.4 ¿Qué diferencia hay, en el formato del documento, entre un caso de uso "referencial" y uno de "movimiento"?

Un referencial es un mantenimiento simple de catálogo (alta/baja/modificación de un dato maestro, ej. Especialidad) y se documenta con bloques Caso Alta / Caso Baja / Caso Modificación, con pasos con letras (a, b, c...).

Un movimiento es una operación de negocio con flujo real (ej. Registrar Agenda Médica, Registrar Factura) y se documenta distinto: bloques Nuevo / Anular (sin la palabra "Caso"), con pasos numerados (1, 2, 3...), y el Flujo Alternativo referencia el paso exacto del bloque (ej. `a.3: si el horario ya está ocupado, el sistema rechaza la operación`).

2.5 ¿Por qué el módulo de Seguridad se corrigió después de redactado?

La primera versión ponía "Mantener Seguridad" como un caso de uso más entre varios. El profesor observó que "Mantener Seguridad" es el nodo raíz del módulo, y que debajo de él hay 5 referenciales reales y distintos: Permisos, Menú, Accesos, Usuarios y Auditoría — no una sola entidad. Esa corrección hizo pasar el índice total de la tesis de 62 a 52 casos de uso documentados.

2.6 Un caso de uso documentado, ¿describe cómo funciona hoy el código o cómo debería funcionar?

Como regla general, el documento describe el sistema como si ya estuviera implementado tal cual se especifica — es un documento normativo de análisis y diseño, no una bitácora de pendientes.

La única excepción manejada explícitamente: en Movimientos, el actor documentado es el que debería tener la operación según el rol de negocio (Especialista/Recepcionista/Administrador), incluso si el control de acceso real en el código hoy es más permisivo — ahí sí se prioriza el diseño deseado sobre el estado actual. Cualquier brecha real entre documento y código va aparte, en la bitácora de auditoría técnica, nunca en el cuerpo del caso de uso.

2.7 ¿Qué es la sección "Prototipo" que se agregó a cada caso de uso?

Una sección nueva, ubicada entre "Lista de archivos" y "Diagrama de secuencia", pensada para incorporar más adelante capturas de pantalla reales de la aplicación corriendo — se deja el título vacío intencionalmente hasta completar esa evidencia visual, sin contenido de relleno.

2.8 ¿Qué diferencia hay entre `<<include>>` y `<<extend>>`?

`<<include>>` es obligatoria: el caso de uso base SIEMPRE ejecuta el incluido como parte de su flujo (ej. todo caso de uso valida sesión). `<<extend>>` es condicional/opcional: el caso de uso extendido solo se ejecuta si se cumple una condición puntual en un punto de extensión definido del caso base (ej. un flujo alternativo que solo dispara bajo cierta circunstancia). La flecha de `<<include>>` sale del caso base hacia el incluido; la de `<<extend>>` sale del caso que extiende hacia el caso base que extiende.

En el `.uml` legacy de esta tesis solo se usan relaciones `<<include>>` (ver 2.2) — no hay ningún `<<extend>>` modelado todavía en los diagramas ya construidos.

2.9 ¿Qué forma tiene siempre un caso de uso en el diagrama?

Un óvalo (elipse), con el nombre del caso de uso adentro o debajo. Es la notación fija de UML para casos de uso, independiente de la herramienta — en StarUML corresponde al elemento `UseCase` del toolbox de diagramas de casos de uso. El actor, en cambio, se dibuja como un muñeco de palitos (stick figure); el rectángulo que engloba a todos los casos de uso representa los límites del sistema.

---

3. Diagrama de clases

3.1 ¿Qué es un diagrama de clases?

El diagrama estructural que muestra las clases del sistema, sus atributos, sus operaciones y las relaciones (asociación, agregación, composición, dependencia, generalización) entre ellas. Es la vista estática por excelencia de UML.

3.2 ¿Qué representa la clase* Conexion *que aparece repetida en todos los diagramas?

Es la clase de control genérica y compartida del formato académico legacy pedido por el profesor: representa la conexión a base de datos, con atributos Host, Usuario, Contraseña, Puerto, BD y una única operación `conectar`. Se reutiliza la misma instancia de modelo en todos los diagramas de clase del proyecto en vez de crear una clase de control distinta por entidad (ej. no existe `EspecialidadControl`).

3.3 ¿Por qué la clase boundary (GUI_X) tiene siempre los mismos 7 botones y 5 operaciones?

Porque el formato legacy de la cátedra define un patrón fijo para toda pantalla de mantenimiento, independiente de la entidad concreta: botones `btn_agregar, btn_grabar, btn_eliminar, btn_modificar, btn_cancelar, btn_consultar, btn_salir`, y operaciones `limpiar_campos, habilitar_campos, desabilitar_campos, habilitar_botones, desabilitar_botones`. El nombre del botón no tiene que reflejar 100% el comportamiento real (ej. dice `btn_eliminar` aunque por debajo sea un soft-delete) — es la convención del template, no una descripción literal del código.

3.4 ¿Qué atributos lleva la clase entity y de dónde salen?

No es un par genérico cod/des: los atributos de la entity son exactamente las columnas de la tabla "Lista de archivos" (con su Alias) del documento de análisis de ese caso de uso — incluyendo el campo de estado. Además lleva una operación `sp_abm` (genérica, para referenciales simples) o `sp_<entidad>` (para entidades con lógica propia más compleja, ej. Paciente, Funcionario).

3.5 ¿Cuándo se usa asociación simple, agregación y composición en este modelo?

Asociación simple: relación estructural sin implicación de dependencia de vida ni de "parte de". Agregación (rombo hueco, `akAggregate`): la entidad propia del movimiento referencia un catálogo independiente (ej. SlotAgenda→Horario, OrdenDetalle→TipoEstudio). Composición (rombo relleno, `akComposite`): relación cabecera→detalle donde el detalle no existe sin la cabecera (Orden→OrdenDetalle).

3.6 ¿Qué es la multiplicidad y aparece en estos diagramas?

La multiplicidad indica cuántas instancias de una clase pueden asociarse con una instancia de la otra (1, 0..1, 0.., *1..*). El formato legacy usado en la tesis prioriza mostrar el tipo de relación (agregación/composición/asociación) y el nombre real de los atributos por sobre anotar la multiplicidad en cada extremo — es un punto que se puede mencionar como mejora si el tribunal pregunta por rigor adicional, pero no es el foco de la convención pedida por la cátedra en este trabajo.

3.7 Diagnóstico tiene CIE-10, Insumo tiene stock: ¿cómo se reflejan esos campos "extra" en el diagrama?

Se agregan como atributos reales adicionales de la clase entity, con el nombre y formato ya usado en el proyecto (`cod_cie10`, `stock_actual`, `stock_minimo`, `insumo_precio_unitario`) — no un genérico `_cod2/_des2`. La regla del profesor fue explícita: si la entidad real tiene 1 o 3 columnas relevantes, esas columnas van sí o sí en el diagrama, igual que en el de secuencia — es parte de la nivelación exigida.

3.8 ¿Por qué ninguna clase de control tiene* sus propios *métodos de negocio (ej. validarFormato, existeDuplicado)?

Porque esa fue justamente la primera versión construida — técnicamente precisa, con una clase de control por entidad y sus métodos reales verificados contra el DAO — y el profesor la corrigió: para la defensa, lo que se evalúa es que el modelo siga la convención de 3 capas de la cátedra con la `Conexion` compartida, no que refleje 1:1 la implementación real en Python/SQL.

---

4. Diagrama de secuencia

4.1 ¿Qué es un diagrama de secuencia y qué muestra que el de casos de uso no muestra?

Muestra el orden temporal exacto de los mensajes que se intercambian los participantes (actor, boundary, control, entity) para realizar un caso de uso — algo que el diagrama de casos de uso no detalla, porque solo dice qué hace el sistema, no paso a paso cómo lo hace internamente.

4.2 ¿Qué es una línea de vida (lifeline) y qué son las activaciones?

La línea de vida es el carril vertical que representa la existencia de un participante a lo largo de la interacción. Las activaciones (los rectángulos angostos sobre la línea de vida) representan el período en que ese participante está ejecutando código en respuesta a un mensaje recibido.

4.3 En cada diagrama de secuencia de esta tesis, ¿qué cuatro participantes aparecen siempre y en qué orden?

Cuatro líneas de vida en un orden fijo: Actor (el rol real de negocio) → `GUI_<ENTIDAD>` (boundary) → `Conexion` (control, compartida) → `<Entidad>` (entity) — reflejando la misma arquitectura de tres capas del diagrama de clases correspondiente.

4.4 ¿Cómo se decide qué mensajes van en cada diagrama de secuencia (Alta, Modificación, Baja)?

El contenido real de los mensajes (validar formato, validar duplicado, ejecutar soft-delete, etc.) se toma del Flujo Básico y Flujo Alternativo verificados contra el código real del caso de uso — no de una plantilla genérica de "abrir ventana → conectar BD → grabar". Esa regla es la que motivó rehacer, uno por uno, los diagramas de secuencia que originalmente venían con contenido genérico heredado del archivo legacy.

4.5 ¿Qué diferencia hay entre "Caso Baja" en un referencial y "Anular" en un movimiento?

En los referenciales no existe eliminación física, solo desactivación (soft-delete) — el bloque se llama "Caso Baja" y la explicación de por qué no hay borrado físico va en la prosa, no en el título. En movimientos, el bloque equivalente se llama simplemente Anular (sin la palabra "Caso"), y representa anular una operación de negocio ya registrada (ej. anular una factura), no un soft-delete de catálogo.

4.6 Un caso de uso tiene un solo diagrama de clases pero varios diagramas de secuencia. ¿Por qué?

El diagrama de clases describe la estructura estática, que no cambia según la operación. El de secuencia describe un flujo de interacción concreto, y cada operación distinta (Alta, Modificación, Baja/Anular) tiene su propia secuencia de mensajes — por eso cada caso de uso referencial tiene 3 diagramas de secuencia y cada movimiento tiene 2 (Nuevo/Anular), todos apuntando al mismo diagrama de clases.

4.7 Si dos entidades similares (ej. Especialidad y Consultorio) tienen diagramas de secuencia casi idénticos, ¿es un error de copiar y pegar?

No necesariamente — es esperable que catálogos simples con el mismo patrón ABM tengan flujos de mensajes muy similares, porque en el código real ambos siguen el mismo patrón de validación/guardado. Lo que sí sería un error es que el texto de los mensajes no se haya actualizado al nombre real de la entidad y sus validaciones específicas (ej. dejar "código de especialidad" en el diagrama de Consultorio)

— ese fue justamente un bug real encontrado y corregido durante la construcción de estos diagramas.

4.8 ¿Qué diferencia hay entre un mensaje síncrono y uno asíncrono en un diagrama de secuencia?

Un mensaje síncrono (flecha llena) espera la respuesta antes de continuar — el emisor queda "bloqueado" hasta recibir el mensaje de retorno (flecha punteada) de vuelta. Un mensaje asíncrono (flecha de línea abierta) se dispara sin esperar respuesta — el emisor sigue su propio flujo en paralelo.

Todos los mensajes de los diagramas de secuencia de esta tesis son síncronos: cada llamada de boundary→control→entity espera la respuesta antes de que la línea de vida emisora continúe (coherente con el patrón de request/response de una aplicación web tradicional). No hay mensajes asíncronos modelados — no correspondería salvo que se documentara algo como el envío de recordatorios por WhatsApp/correo de Agendamiento en un diagrama dedicado a ese proceso en background.

---

5. Diagrama de actividades

5.1 ¿Qué es un diagrama de actividades y en qué se diferencia de uno de secuencia?

Modela el flujo de trabajo (workflow) de un proceso de negocio, con foco en decisiones, ramas paralelas y quién ejecuta cada paso — a diferencia del diagrama de secuencia, que modela mensajes entre objetos de software dentro de un caso de uso puntual. El de actividades tiene una granularidad más de "proceso de negocio" que de "interacción técnica".

5.2 ¿Qué elementos del toolbox de StarUML se usaron realmente y cuáles no existen?

Se usaron: InitialState, ActionState, FinalState, Synchronization (un único elemento que cubre fork Y join, no hay dos separados), Decision, Flow Final y Swimlane. No existen variantes separadas de fork/join, ni forma de anidar sub-carriles, ni formas personalizadas — cualquier rediseño propuesto tiene que poder construirse solo con esa lista.

5.3 ¿Cuándo corresponde usar* Decision *y cuándo* Synchronization*?

Decision se usa solo cuando hay una bifurcación real evidenciada en el código (un `if` que cambia el camino según una condición de negocio) — se evita dibujar diamantes de decisión "ficticios" sin correlato real. Synchronization se usa para ramas paralelas u opcionales que no dependen de una condición evaluada (ej. dos acciones que ocurren a la vez, o el cierre de ramas concurrentes).

5.4 ¿Por qué en el diagrama de Agendamiento el actor Especialista se reemplazó por Sistema en una piscina?

Porque, aplicando la regla de agencia (una piscina debe representar a quien realmente ejecuta la acción), se determinó que Especialista no tenía ninguna acción propia en ese tramo del proceso — las acciones ahí eran automáticas (validaciones o jobs del sistema), así que la piscina correcta era Sistema, no un rol humano sin agencia real en ese punto.

5.5 ¿Cuántas piscinas tiene, por ejemplo, el diagrama de Agendamiento y por qué ese número es fijo?

Tres piscinas. El tutor pidió una cantidad fija de piscinas por diagrama de actividades del módulo, así que cualquier decisión de "qué actor va en cada piscina" se resuelve reemplazando candidatos entre sí (ver 5.4) en vez de agregar o quitar piscinas — el número de piscinas no cambia, cambia quién ocupa cada

una según la evidencia real de agencia en el código.

5.6 ¿Qué diferencia hay entre Flow Final *y* FinalState*?*

Flow Final cierra una rama específica del flujo sin terminar el proceso completo (útil cuando una rama alternativa simplemente deja de continuar, ej. un rechazo de validación). FinalState cierra el proceso completo — se usa una sola vez, al final real de todo el diagrama de actividades.

5.7 ¿Cómo se usa correctamente la barra de sincronización cuando "entra una condición y salen dos respuestas" y viceversa?

La barra de sincronización (`Synchronization`) es simétrica en sus dos usos: como **fork** (división), una única transición de entrada se abre en dos o más transiciones de salida que corren en paralelo — todas las ramas se activan a la vez. Como **join** (unión), dos o más transiciones de entrada convergen en la barra y de ahí sale una única transición — pero el flujo no continúa hasta que TODAS las ramas de entrada llegaron (esto es la "disparidad de AND": a diferencia de una `Decision`, donde solo se toma un camino, en la barra de sincronización deben cumplirse sí o sí todas las condiciones/ramas de entrada antes de liberar la salida).

En esta tesis se usa, por ejemplo, para modelar acciones que ocurren en paralelo dentro de un mismo proceso (ver 5.3) — nunca para representar una decisión real de negocio, que corresponde a `Decision`.

5.8 ¿Cómo se relacionan las validaciones del flujo básico con el flujo alternativo?

Toda validación que se ejecuta en el flujo directo/básico (ej. "el sistema valida que el nombre de usuario no esté en uso") debe tener su contraparte explícita en el flujo alternativo, describiendo qué pasa cuando esa validación falla (ej. "nombre de usuario ya en uso: el sistema rechaza la operación"). Es una regla de consistencia documental: el flujo alternativo no es una lista libre de errores, sino el reverso punto por punto de cada validación mencionada en el básico — si se agrega una validación nueva al flujo básico de un caso de uso, hay que agregar su rechazo correspondiente en el alternativo (y viceversa: no debe haber una condición de rechazo en el alternativo que no tenga su validación mencionada en el básico).

---

6. StarUML y el archivo .uml

6.1 ¿Qué es StarUML y qué formato de archivo usa esta tesis?

StarUML es la herramienta CASE (Computer-Aided Software Engineering) usada para dibujar y mantener todos los diagramas UML del proyecto. El archivo de la tesis (`SYSCOMCLI - version final.uml`) es un formato legacy de StarUML: XML con namespace XPD, no el formato moderno `.mdj` (JSON) de las versiones actuales de la herramienta.

6.2 ¿Por qué un diagrama UML es más que "un dibujo"? — la diferencia entre modelo y vista

Cada elemento (una clase, un caso de uso, un mensaje) existe UNA vez como elemento de modelo (con su nombre y sus relaciones lógicas), y por separado tiene una o más vistas (la representación visual dentro de un diagrama concreto), enlazadas entre sí por referencia. Un mismo elemento de modelo puede aparecer dibujado en varios diagramas distintos, o incluso no tener ninguna vista dibujada todavía ("huérfano").

6.3 ¿Por qué la clase Conexion *aparece como una sola instancia compartida y no 17 copias?*

Porque en el modelo de StarUML, reutilizar el mismo elemento de modelo en varios diagramas de clase (referenciándolo desde cada vista) es semánticamente distinto de crear 17 clases separadas con el mismo nombre. La decisión académica fue que solo existe una clase `Conexion` en todo el proyecto, y los 17+ diagramas de clase apuntan todos a esa misma instancia de modelo — igual que en programación, reutilizar una clase real en vez de duplicarla.

6.4 Si dos diagramas de clase muestran la misma entidad con atributos distintos, ¿cómo se explica?

No debería pasar: como la clase vive una sola vez en el modelo y las vistas solo la referencian, cambiar sus atributos los actualiza en todos los diagramas donde aparece dibujada. Si se ve una discrepancia, es señal de que en algún momento se creó una copia nueva del modelo en vez de reutilizar la existente (un error de construcción, no una decisión de diseño).

6.5 ¿Por qué la asociación cabecera→detalle necesita un rombo relleno y no alcanza con dibujar una línea?

Porque en UML el tipo de relación (composición vs. agregación vs. asociación simple) es información semántica del modelo, no un estilo visual: define si la parte puede vivir sin el todo. Una línea sin rombo en el diagrama se interpreta como asociación simple, lo cual sería semánticamente incorrecto para, por ejemplo, Orden→OrdenDetalle (que sí depende de su cabecera para existir).

6.6 ¿Todos los diagramas de secuencia de la tesis vienen "de fábrica" con las clases correctamente vinculadas al diagrama de clases?

No automáticamente — hay que verificarlo: una línea de vida en un diagrama de secuencia debe tener su Clasificador (la propiedad que la vincula a la clase real del diagrama de clases correspondiente) correctamente asignado. Sin esa referencia, la línea de vida es solo una caja con una etiqueta de texto, sin relación real de modelo con la clase — StarUML no "sabe" que esa línea de vida representa, por ejemplo, un objeto real de `GUI_ESPECIALIDAD`.

6.7 Para la defensa: si el tribunal pregunta "¿por qué las clases boundary/control/entity no coinciden con los nombres reales de las clases Python del proyecto?" — ¿qué se responde?

Porque el diagrama de clases de este capítulo sigue la convención académica de análisis orientado a objetos (boundary/control/entity) tal como se enseña en la cátedra, que es un nivel de abstracción distinto y anterior al diseño de implementación real (los módulos, DAOs y rutas Flask del proyecto). Ambos documentos son válidos y coexisten: el capítulo de análisis usa el lenguaje de la cátedra; el código usa el lenguaje de la arquitectura de implementación real del sistema.

---

7. Diagrama de despliegue

> Nota: este diagrama todavía no está construido en la tesis (ver `pendientes_taller_20260818.md`, punto 5). Las siguientes son preguntas puramente teóricas para tenerlas resueltas antes de encararlo.

7.1 ¿Qué es un diagrama de despliegue y qué muestra que ningún otro diagrama muestra?

Es el único diagrama UML que representa la topología física real de hardware donde corre el sistema: qué nodos (servidores, equipos cliente) existen, qué software se instala en cada uno y cómo se comunican entre sí. A diferencia de los demás diagramas (que son 100% lógicos/de software), el de

despliegue es el puente entre el diseño y la infraestructura real.

7.2 ¿Qué es un nodo y qué es un artefacto?

Un **nodo** (`Node`) es un elemento de hardware o de entorno de ejecución (un servidor físico o virtual, un equipo cliente, un dispositivo) — se dibuja como un cubo/prisma 3D. Un **artefacto** (`Artifact`) es la pieza de software concreta que se despliega dentro de un nodo (un archivo ejecutable, un paquete, una base de datos, un script) — se dibuja como un rectángulo con el ícono de documento con esquina doblada. La relación entre ambos es de contención: el artefacto "vive dentro" del nodo.

7.3 ¿Cómo se conectan los nodos entre sí?

Con una asociación simple (una línea, a veces etiquetada con el protocolo de comunicación, ej. HTTP/TCP-IP) llamada **camino de comunicación** (`communication path`). No lleva flecha de dirección obligatoria — representa el enlace físico/lógico entre los dos nodos, no un flujo de mensajes como en el diagrama de secuencia.

7.4 Para SYSCOMcli, ¿qué nodos y artefactos correspondería modelar como mínimo?

Como mínimo: un nodo servidor de aplicación (donde corre la app Flask), un nodo servidor de base de datos (PostgreSQL) — pueden ser el mismo nodo físico o dos distintos según el despliegue real — y un nodo cliente genérico (el navegador del usuario). Como artefactos: el ejecutable/paquete de la aplicación Flask dentro del nodo de aplicación, y la base de datos `sysclin_bd` dentro del nodo de base de datos. Esto es un punto de partida a confirmar contra el despliegue real del proyecto antes de darlo por definitivo.